Date: 30/7/26

Task: 1 Implementation of a Binary classification model for grey-scale image

Aim:-

To implement a binary classification model for grayscale images using pytorch.

Algorithm:-

1) Import the required libraries.
2) Load the grayscale image dataset.
3) Resize images and convert them into tensors.
4) Build a neural network with input, hidden, and output layers.
5) Train the model using a loss function and optimizers.
6) Predict the class of the input image.

Program

```
import torch
import torch.nn as nn
import torch.optim as optim
X = torch.randn(100, 784)
y = torch.randint(0, 2, (100, 1)).float()
```

Output:

Training completed

predicted class: 0

```
model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Sigmoid(),
)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(),
                       lr=0.001)
for epoch in range(10):
    optimizer.zero_grad()
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
print("Training completed")
print("Predicted class:", int(model(X[:1]).item()>=0.5))
```

VEL TECH - IT	
EX NO.	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	

Result: Thus, a binary classification model for grayscale image data was implemented using pytorch.

Date: 6/8/26

Task: 2 Convolutional Neural Network classification

Aim: To implement an image classification model using a convolutional Neural Network in PyTorch.

Algorithm

1) Import PyTorch libraries.
2) Load image data.
3) Define convolution, pooling, and fully connected layers.
4) Apply ReLU activation.
5) Train the CNN using a loss function and optimizer.
6) Predict the image class.

Program:

```
import torch
import torch.nn as nn
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, 3)
        self.pool = nn.MaxPool2d(2)
```

Output

CNN output shape : torch.Size([1, 10])

Predicted class : 3

```
    def forward(self, x):
        x = self.pool(torch.relu(self.conv(x)))
        x = x.view(x.size(0), -1)
        return self.fc(x)

model = CNN()
x = torch.randn(1, 1, 28, 28)
output = model(x)
print("CNN output shape:", output.shape)
print("Predicted class:",
output.argmax(dim=1).item())
```

VEL TECH - IT	
EX NO.	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	4
TOTAL (20)	19
SIGN WITH DATE	

Result: Thus, a CNN model for image classification was implemented using PyTorch.

Date 13/8/26

Task: 03 Construct a ConvNet Model for Image Classification

Aim: To construct a Convolutional Network for image classification using pytorch.

Algorithm

1) Import the required libraries.
2) Define convolutional and pooling layers.
3) Flatten the feature maps.
4) Add fully connected layers.
5) Pass an image through the model.
6) Display the predicted class.

Program:

```
import torch
import torch.nn as nn
class ConvNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.RELU(),
```

output

CONVNet created successfully

output shape: torch.Size([1, 10])

```
        nn.MaxPool2d(2).
        )
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(32*7*7, 10)
    )
def forward(self, x):
    return self.classifier(self.features(x))
model = ConvNet()
x = torch.randn(1, 1, 28, 28)
output = model(x)
print("ConvNet created successfully")
print("output shape:", output.shape)
```

VEL TECH - IT	
EX NO.	3
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	4
TOTAL (20)	19
SIGN WITH DATE	

Result:- Thus, a ConvNet model for image classification was constructed using pytorch.

Date: 20/8/26

Task: 4 Transfer learning for Image classification

Aim:

To apply transfer learning to build an image classification model using pytorch.

Algorithm

1) Import pytorch and torchvision.
2) Load a pretrained ResNet-18 Model.
3) Freeze the pretrained feature layers.
4) Replace the final layer with a new classification layer.
5) Pass an input image through the model.
6) Display the output shape.

Program:

```
import torch
import torch.nn as nn
From torchvision import models
model = models.resnet18(weights= None)
for param in model.parameters():
Model.fc = nn.Linear(model.fc.in_features, 2)
x = torch.randn(1, 3, 224, 224)
    output = model(x)
print("Transfer learning model created")
print("output shape:", output.shape)
```

Output:

Transfer learning model created

output shape : ~~torch~~.Size([1,2])

VEL TECH - IT	
EX NO.	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	

Result: Thus, a transfer learning model for image classification was implemented using pyTorch.

Date:- 3/9/26

Task:- 5 Simple Generative Adversarial Network

Aim:- To implement a simple generative adversarial network using pytorch.

Algorithm:-

1) Import the required libraries.
2) Create a generator network.
3) Create a discriminator network.
4) Generate random noise as input to the generator.
5) Train the discriminator to distinguish real and fake data.
6) Train the generator to produce realistic data.

Program

```
import torch
import torch.nn as nn
generator = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
    nn.Tanh()
)
discriminator = nn.Sequential(
    nn.Linear(1, 32),
    nn.ReLU(),
```

Gan model created successfully.

Generated data shape : torch.Size([5, 1])

Discriminator output shape : torch.Size([5, 1])

```
        nn.Linear(32, 1),
        nn.Sigmoid()
    )

print("GAN model created successfully")
print("Generated data shape:", fake_data.shape)
print("Discriminator output shape:", output.shape)
```

VEL TECH - IT	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	19
SIGN WITH DATE	

Result: Thus, a simple GAN Consisting of a generator and discriminator was implemented using Pytorch.

Date: 16/9/26

Task: 6 Word Embedding Using RNN

Aim: To implement word embedding and sequence processing using an RNN in PyTorch.

Algorithm:

1) Import PyTorch libraries.
2) Define a vocabulary.
3) Convert words into numerical indices.
4) Create an embedding layer.
5) Pass the embedded sequence through an RNN.
6) Display the output shape.

Program

```
import torch
import torch.nn as nn
vocab_size = 10
embedding_dim = 8
hidden_size = 16
embedding = nn.Embedding(vocab_size, embedding_dim)
batch_first = True)
words = torch.tensor([[1, 2, 3, 4]])
embedded = embedding(words)
output, hidden = rnn(embedded)
```

output

Embedding output shape : torch.Size([1, 4, 8])

RNN output shape : torch.Size([1, 4, 16])

```
print("Embedding output shape :", embedded.shape)
print("RNN output shape :", output.shape)
```



VEL TECH - IT	
EX NO.	6
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	

Result :- Thus, Word embedding and RNN sequence processing were implemented using PyTorch.

Date: 7/9/26

Task: 7 Implementation of Autoencoders

Aim: To implement a simple autoencoder using pytorch.

Algorithms:

1) Import the required libraries.
2) Define the encoder network.
3) Define the decoder network.
4) pass input data through the encoder.
5) Reconstruct the input using the decoder.
6) Display the reconstructed output shape.

Program

```
import torch
import torch.nn as nn
class Autoencoder (nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(32, 128),
            nn.ReLU
            nn.Linear(128, 784),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

Output:-

Autoencoder created successfully

Reconstructed shape: torch.Size([1, 784])

```
model = Autoencoder()
X = torch.rand(1, 784)
reconstructed = model(X)
print("Autoencoder created successfully")
print("Reconstructed shape:", reconstructed.shape)
```

VEL TECH - IT	
EX NO.	7
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	

Result: Thus, an autoencoder was implemented to encode and reconstruct input data using PyTorch.

Date: 24/9/26

Task:8 Prediction Using LSTM Networks

Aim: To implement sequence prediction using an LSTM network in PyTorch.

Algorithm:

1) Import PyTorch libraries.
2) Create sequential input data.
3) Define an LSTM layer.
4) Pass the sequence through the LSTM.
5) Use a fully connected layer to predict the next value.
6) Display the prediction.

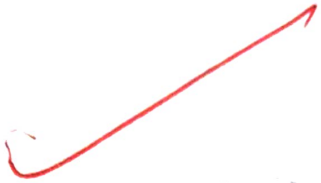
Program

```
import torch
import torch.nn as nn
class LSTMModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(1, 16, batch_first=True)
        self.fc = nn.Linear(16, 1)

    def forward(self, x):
        output, _ = self.lstm(x)
        return self.fc(output[:, -1, :])
```

Output:

LSTM prediction : 0.12



```
prediction = model(x)
print("LSTM prediction:", prediction.item())
```

VEL TECH - IT	
EX NO.	8
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	

Result:- Thus, sequence prediction using an LSTM network was implemented using PyTorch.

Date: 8/10/26

Task: 9 Object Detection and Localization Using OpenCV

Aim:

To implement basic object detection and localization using opencv.

Algorithm:

1. Import the Open cv library.
2. Load an image.
3. Convert the image to grayscale.
4. Apply thresholding to identify the object.
5. Find the object's contours.
6. Draw a bounding box around the detected object.

Program

```
import cv2
img = cv2.imread("image.jpg")
if img is None;
  print("image not found")
else:
  gray = cv2.cvtcolor(img, cv2.colour_BGR2
                                GRAY)
  )
  Contours, _ = cv2.findContours(
    thresh, cv2.RETR_EXTERNAL,
    CV2.CHAIN_APPROX_SIMPLE
  )
```

Output :-

Object detection completed

Output image saved as output.jpg.

```
for contour in contours:
    if cv2.contourArea(counter) > 100:
    x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(
            img, (x, y), (x+w, y+h),
                (0, 255, 0), 2
        )
cv2.imwrite("output.jpg", img)
print("object detection completed")
```

VEL TECH - IT	
EX NO.	9
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	

Result:-
Thus, basic object detection and localization were implemented using opencv.

Date:- 15/10/26

Task:- 10 Optimization Techniques to improve Model

Aim:- To apply optimization techniques to improve neural network performance using Pytorch.

Algorithm:-

1. Import PyTorch libraries.
2. Create input data and target values.
3. Define a neural network.
4. select the Adam optimizer and mean squared error loss.
5. Train the model for multiple epochs.
6. Display the final loss.

Program

```
import torch
import torch.nn as nn
import torch.optim as optim.

X = torch.randn(100, 1)

Y = 2 * X + 1

model = nn.Linear(1, 1)

criterion = nn.MSELoss()

for epoch in range(100):
    prediction = model(X)

loss = criterion(prediction, Y)

optimizer, zero_grad()
```

output

optimization Completed

Final loss: A small value close to 0

```
        loss.backward()
        optimizer.step()
print("optimization completed")
print("final loss:", loss.item())
```

VEL TECH - IT	
EX NO.	10
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	4
TOTAL (20)	19
SIGN WITH DATE	

Result:-

Thus, the Adam optimization technique was applied to train a neural network and reduce its loss.

Date: 22/10/25

Usecase 5 :- Time Series Forecasting

Aim: To implement time series forecasting using an LSTM network in pytorch.

Algorithm

1) Import the required libraries.
2) Create time series data.
3) prepare the input sequences and target values.
4) Define an LSTM model.
5) Train the model using a loss function and optimizer.
6) predict the next value in the time series

Program :-

```
import torch
import torch.nn as nn
import torch.optim as optim

data = torch.arange(1, 21, dtype = torch.float32)

X, Y = [], []

for i in range(len(data) - 3):
    X.append(data[i:i+3])
    Y.append(data[i+3])

Model = nn.LSTM(1, 16, batch_first = True)

fc = nn.Linear(16, 1)

optimizer = optim.Adam(
```

output

Predicted next value : Approximately 21.00

```
    lr = 0.01
)
criterion = nn.MSELoss()

for epoch in range(100):
    out, _ = model(x)
    pred = fc(out[:, -1, :])
    loss = criterion(pred, y)
optimizer.zero_grad()
loss.backward()
optimizer.step()
out, _ = model(test)
prediction = fc(out[:, -1, :])
print("Predicted next value:",
      round(prediction.item(), 2))
```

VEL TECH - IT	
EX NO.	11
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	19
SIGN WITH DATE	

Result:

Thus, time series forecasting was implemented successfully using an LSTM network in PyTorch.

Date :- 22/10/25

Usecase 6: Car Detection using YOLO

Aim: To detect cars in an image using the YOLO.

Algorithm

1) Install and import the Ultralytics YOLO library.
2) Load a pretrained YOLO model.
3) Read the input image.
4) Perform object detection using YOLO
5) Identify cars and their bounding boxes.
6) Display and save the detected image.

Program

```
import YOLO
import CV2
model = YOLO("yolov8n.pt")
image = cv2.imread("cars.jpg")
if image is None:
    print("Image not found")
else:
    results = model(image)
    for result in results:
        for box in result.boxes:
            class_id = int(box.cls[0])
if model.names[class_id] == "car":
```

output

Car detection completed

Output saved as detected_cars.jpg

```
x1, y1, x2, y2 = map(
    int, box.xyxy[0]
)

cv2.rectangle(
    image, (x1, y1), (x2, y2),
    (0, 255, 0), 2
)

cv2.putText(
    image, "car",
    (x1, y1-10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.7, (0, 255, 0), 2
)

cv2.imwrite("detected_cars.jpg", image)
print("Car detection completed")
print("Output saved as detected_cars.jpg")
```

VEL TECH - IT	
EX NO.	12
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	

Result

Thus the program was executed and output got verified.